



Recitation 10

Lecture overview

- The vector object
- The `FILE` object for file I/O

Lecture overview

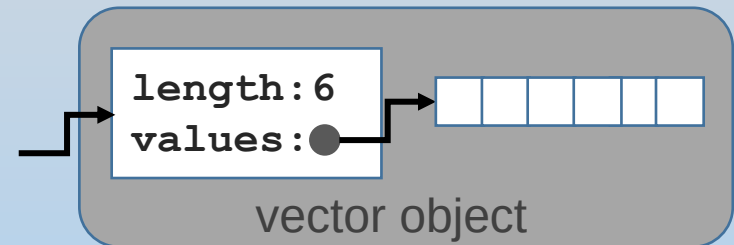
- The vector object ←
- The `FILE` object for file I/O

The Vector “class” types

- The struct is used to “attach” the array (of `float`) to its length to prevent access beyond array boundaries
- The array is not part of the structure. This will enable vectors of unbounded sizes.

The structure type:

```
typedef struct {  
    int length;  
    float* values;  
} Vector_st;
```



The pointer type:

```
typedef Vector_st* Vector;
```

- This is the main type used by functions that involve vector objects

The Vector “class”

```
/** structure type */
typedef struct {
    int length;
    float* values;
} Vector_st;
typedef Vector_st* Vector;

/** allocate memory for vector object (incl. array) */
Vector newVector(int len);

/** free all memory allocated by vector object */
void freeVector(Vector v);

/** set one vector element */
int setVectorElement(Vector v, int ind, float val);

/** set a block of vector elements */
int setVectorBlock (Vector v,
    int startInd, int numElements, float* source);

/** return one vector element */
float getVectorElement(Vector v, int ind);

/** compute sum of vector entries */
float sumVector(Vector v);

/** print vector entries */
void printVector(Vector v);
```

The vector “class” constructor

Allocate vector object on the heap (structure + array):

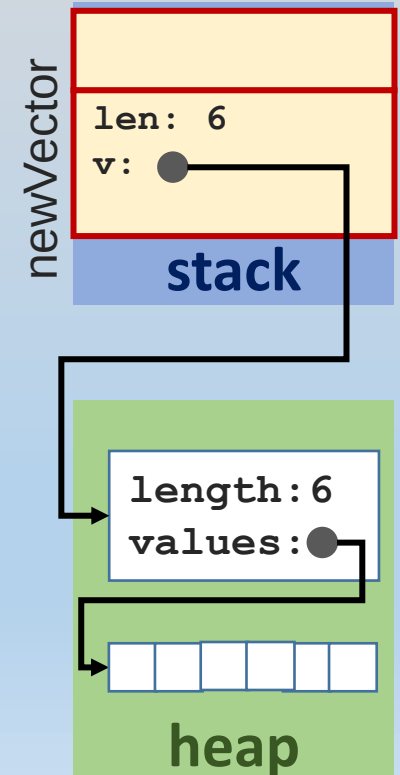
```

Vector newVector(int len) {
    Vector v;
    if(len <= 0) return NULL;

    v = malloc(sizeof(Vector_st));
    if(!v) return NULL;

    v->values = malloc(len*sizeof(float));
    if(!v->values) {
        free(v);
        return NULL;
    }
    v->length = len;
    return v;
}
    
```

We discussed this in detail in Lecture #10



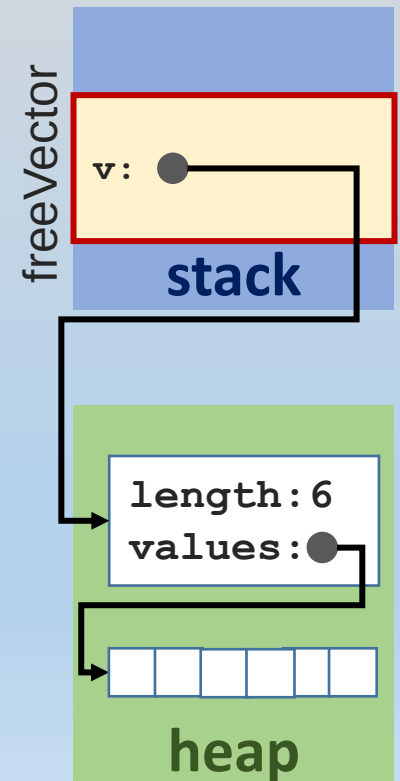
The vector “class” destructor

The destructor is responsible to free object (structure + array):

```
void freeVector(Vector v) {  
    free(v->values);  
    free(v);  
}
```

- The space allocated to vector object includes structure and array
- First free array and then free structure

We discussed this in detail in Lecture #10



The Vector “class” methods (functions)

Read vector element:

```
float getVectorElement(Vector v, int ind) {  
    if(ind<0 || ind >= v->length) return 0.0;  
    return v->values[ind];  
}
```

- Receives reference (pointer) to vector struct and index
recall: `typedef Vector_st* Vector;`
- If index is out-of-bounds, return 0.0
- Otherwise, return array element

The Vector “class” methods (functions)

Compute vector sum:

```
float vectorSum(Vector v) {  
    int i;  
    float sum=0;  
    for(i=0; i<v->length; i++) {  
        sum += v->values[i];  
    }  
    return sum;  
}
```

- Receives reference (pointer) to vector struct
- Computes vector sum and returns it

The Vector “class” methods (functions)

Print vector:

```
void vectorPrint(Vector v) {  
    int i;  
    printf("[");  
    for(i=0; i<v->length; i++) {  
        printf(" %.3f",v->values[i]); /** 3 decimal places */  
    }  
    printf(" ]");  
}
```

- Receives reference (pointer) to vector struct
- Prints vector in the following format (no newline):

```
[ 0.003 0.210 34.100 ]
```

[%.3f – print 3 digits after decimal point]

The Vector “class” methods (functions)

Write vector element:

```
int setVectorElement(Vector v, int ind, float val) {  
    if(ind<0 || ind >= v->length) return 0;  
  
    v->values[ind] = val;  
    return 1;  
}
```

- Receives reference (pointer) to struct vector, index, and value
- If index is out-of-bounds, return 0 (to indicate error)
- Otherwise, write element and return 1

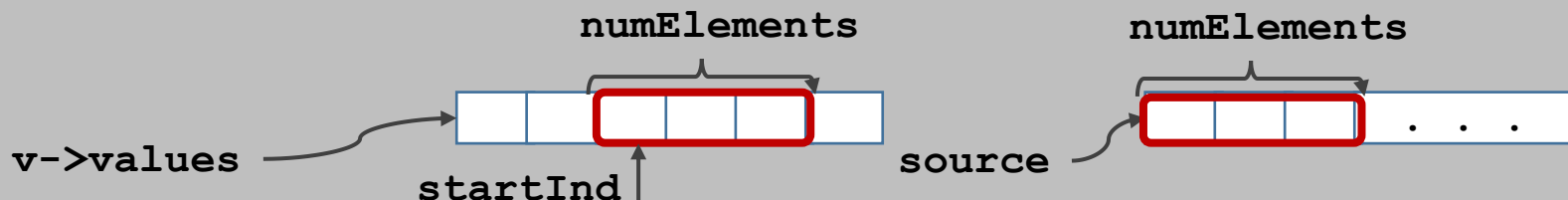
The Vector “class” methods (functions)

Write block of vector elements:

```

int setVectorBlock (Vector v,      int startInd,
                    int numElements, float* source) {
    int ind;
    if(startInd < 0 || numElements <= 0 ||
        startInd + numElements > v->length)
        { return 0; }
    for(ind=startInd; ind < startInd+numElements; ind++) {
        v->values[ind] = source[ind - startInd];
    }
    return 1;
}
  
```

- Copy a block of `numElement` values from the start of a given `source` array to the vector starting at index `startInd`



The Vector “class” methods (functions)

Write block of vector elements:

```
int setVectorBlock (Vector v,      int startInd,
                    int numElements, float* source) {
    int ind;
    if(startInd < 0 || numElements <= 0 ||
        startInd + numElements > v->length)
        { return 0; }
    for(ind=startInd; ind < startInd+numElements; ind++) {
        v->values[ind] = source[ind - startInd];
    }
    return 1;
}
```

- Copy a block of **numElement** values from the start of a given **source** array to the vector starting at index **startInd**
- If indices are out of bound, do nothing and return 0
- Otherwise, copy block and return 1

The Vector “class” – usage

```

int main() {
    Vector v1, v2, v3;
    /** vals = { 1.0 , 0.0 , 0.0 , ... , 0.0 }***/
    float vals[100] = {1.0};
    int i;
    /** set vals[i] = 1 / 2^i ***/
    for (i=1; i< sizeof(vals)/ sizeof(vals[0]); i++)
        { vals[i] = 0.5 * vals[i-1];    }

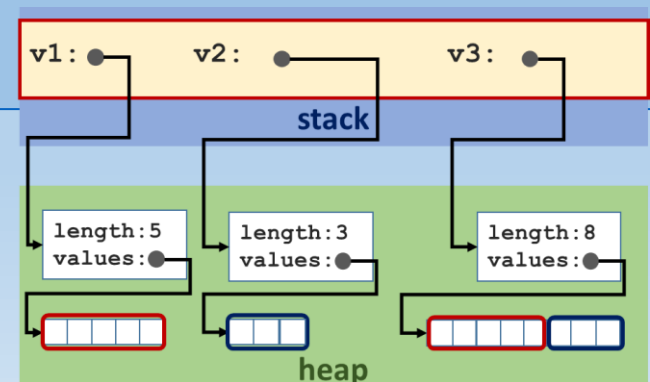
    . . .

    freeVector(v1);
    freeVector(v2);
    freeVector(v3);

    return 0
}
    
```

See: `/share/recitations/recitation10/vector.c`

Class exercise: add code to main that will create a vector object v3 that is a concatenation of v1 and v2



Lecture overview

- The vector object
- The **FILE** object for file I/O ←

The FILE object

- The `stdio` library has various functions for working with files
- These functions use the `FILE` structure to hold information on a file (read/write status, current location in file, etc.)
- The internal structure of `FILE` is kept “private”.
- We use the `FILE` object through a list of interface functions.
- A `FILE` object can also be used to reference standard I/O streams (`stdin`, `stdout`, and `stderr`)

The FILE object

Opening a file:

```
FILE* fopen(const char* filename, const char* mode);
```

parameters:

filename - string containing path of file to be opened

mode - string indicating mode of opening

"r" – read / "w" – write / "a" – append

return:

pointer to **newly allocated** FILE object (stream)

NULL if failed (e.g., file does not exist / no permissions / no space to allocate structure on heap)

The FILE object

Closing a file:

```
int fclose(FILE* stream);
```

parameters:

file - pointer to FILE object (stream)

return:

0 if successful

EOF (-1) if failed

Function closes stream and frees file object

The FILE object

Standard streams:

- `stdin`, `stdout`, and `stderr` are constants of type `FILE*` defined in `stdio` which refer to standard i/o/e streams
- No need to open/close these streams – nothing happens when we perform `fclose(stdin)`

The FILE object

Printing into file:

```
int fprintf(FILE* stream, const char* format, ...);
```

- Same format as `printf()`, but with additional stream param
- Example usage – print **Hello** to a file called **file.txt**:

```
FILE* f;  
f = fopen("file.txt", "w");  
fprintf(f, "Hello\n");  
fclose(f);
```

```
printf("Hello world %d\n", 2021);
```

is the same as:

```
fprintf(stdout, "Hello world %d\n", 2021);
```

Printing to file – example

Print “Hello world!” into a file specified as argument.

```
#include <stdio.h>

int main(int argc, char** argv) {
    FILE* hello_file;
    char* fileName;
    if(argc<2) {
        fprintf(stderr, "Usage: %s file\n" , argv[0]);
        return 1;
    }
    fileName = argv[1];

    /*** open file and check ***/
    hello_file = fopen(fileName,"w");
    if(!hello_file) {
        fprintf(stderr, "Unable to open file %s for writing\n", fileName);
        return 1;
    }

    /*** print to file and close ***/
    fprintf(hello_file,"Hello world!\n");
    fclose(hello_file);
    return 0;
}
```

← make sure program has expected num of arguments!

The FILE object

Reading from file:

```
int fscanf(FILE* stream, const char* format, ...);
```

- Same format as **scanf()**, but with additional stream param

```
scanf("%d %lf\n", &myInt, myNum);
```

is the same as:

```
fscanf(stdin, "%d %lf\n", &myInt, myNum);
```

- **scanf()** can be problematic, because of its behavior when encountering input that doesn't follow expected format

The FILE object

One of the most common and safe ways to read information from files is by reading line by line into a pre-allocated string using `fgets()`

```
char *fgets (char *line, int n, FILE *stream)
```

- Read characters from input stream until reaching:
 - the next newline character (' \n ') (including)
 - The end of the file (EOF)
 - **n-1** characters
- Write the read characters into the buffer pointed to by **str**
- Terminate written string with ' \0 ' character
- If the end of the file was reached, returns NULL
- Otherwise, returns the pointer **str** (to the beginning of written buffer)

The FILE object

One of the most common and safe ways to read information from files is by reading line by line into a pre-allocated string using `fgets()`

```
char *fgets (char *line, int n, FILE *stream)
```

- Read characters from input stream until reaching:
 - the next newline character (' \n ') (including)
 - The end of the file (EOF)
 - n-1 characters

Notes:

- Buffer overflow can be avoided if we pass buffer length as n.
- If a line contains less than n-1 characters, then the written string contains the '\n' character before the terminating '\0'.
- Function returns NULL when reaching EOF. Typically, the last line in the file ends with a newline, and then, EOF is reached when reading an empty line.

Reading complete lines – example

- Add line numbers to standard input stream:

```
#include <stdio.h>

#define MAX_LINE_SIZE 80

int main() {

    char buf[MAX_LINE_SIZE];
    int line = 1;

    while (fgets (buf, MAX_LINE_SIZE, stdin) != NULL)
        printf ("%4d  %s", line++, buf);

    return 0;
}
```

Reading from file – example

Read file and print each line with its index + number of characters

File header + main (part 1 – variable definition)

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

/** buffer size - max line length + newline + '\0' */
#define BUFFER_SIZE 202

int main(int argc, char** argv) {
    /** buffer for holding line from file */
    char line[BUFFER_SIZE];

    int nLines=0, nChars, c;
    FILE* inFile;
    . . .
```

See `/share/recitations/recitation10/lineStats.c`

Reading from file – example

Read file and print each line with its index + number of characters
main (part 2 – check argument)

```
. . .  
/** check one argument, and open file + validate **/  
if(argc!=2) {  
    fprintf(stderr,"Usage: %s <file>\n",argv[0]);  
    return 1;  
}  
inFile=fopen(argv[1],"r");  
if(!inFile) {  
    fprintf(stderr,"Cannot open file %s for reading\n",argv[1]);  
    return 1;  
}  
. . .
```

See `/share/recitations/recitation10/lineStats.c`

Reading from file – example

Read file and print each line with its index + number of characters
 main (part 3 – reading loop)

```

. . .
  /*** main loop - read file from stdin line by line ***/
  while(fgets(line,BUFFER_SIZE,inFile)) {
    nLines++;
    nChars=strlen(line);

    /*** print line with stats ***/
    printf("%3d (%3d chars): %s", nLines, nChars, line);

    /*** if line exceeds LINE_LEN, then indicate truncation ... ***/
    if(line[strlen(line)-1] != '\n') {
      c=0;
      while(c!=EOF && c!='\n') {
        c=fgetc(inFile);
      }
      printf("|trunc at %d\n", BUFFER_SIZE-1);
    }
  } // end of while(fgets)
. . .

```

See `/share/recitations/recitation10/lineStats.c`

Reading from file – example

Read file and print each line with its index + number of characters
main (part 4 – conclusion)

```
. . .  
    /** close file and return **/  
    fclose(inFile);  
    return 0;  
}  
. . .
```

Reading/writing single characters

- Read next character from file:

```
int fgetc (FILE *stream)
```

- return **EOF** when file is done or other failure

- Write character to file:

```
int fputc (int c, FILE *stream)
```

- return **EOF** for failure (file not open for writing)

Notes:

- **stdin**, **stdout** and **stderr** are constants of type **FILE*** defined in **stdio.h** which refer to standard input / output / error streams
- Functions **putchar** and **getchar** are equivalent to **fputc** and **fgetc**, but work directly with **stdout** and **stdin**

File read / write – example

- Read input file and write capital letter counts in output file

```
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char *argv[]) {
    int    c, i, letterCount[26]={0}; ← locally allocated array
    FILE *ifp, *ofp;                  ← input / output file streams

    if (argc != 3) {                  ← check two input arguments
        fprintf (stderr,              ← error message printed to stderr
            "\nUsage: %s infile outfile\n\n", argv[0]);
        fprintf (stderr,
            "    Counts uppercase letters in infile.\n");
        fprintf (stderr,
            "    Writes the results to outfile.\n\n");
        return 1;
    }
    ...
}
```

File read / write – example

- Read input file and write capital letter counts in output file

```
ifp = fopen (argv[1], "r");  ← open for reading
if(!ifp) {
    fprintf(stderr,
        "File %s could not be open for reading", argv[1]);
    return 1;
}
ofp = fopen (argv[2], "w");  ← open for writing
if(!ofp) {
    fprintf(stderr,
        "File %s could not be open for writing", argv[2]);
    fclose(ifp);              ← remember to close when aborting (free memory)
    return 1;
}

while ((c = fgetc (ifp)) != EOF) {
    if (c >= 'A' && c <= 'Z') {
        letterCount[c - 'A']++;  ← read characters one at a time and
                                   update counts for letters
    }
}
...
```


File read / write – example

- Read input file and write capital letter counts in output file

```
for (i = 0; i < 26; i++) {    ← print count for every letter in output file
    if (i % 6 == 0)          ← print 6 letters in line
        fputc ('\n', ofp);
    fprintf (ofp, "%c:%4d| ", 'A' + i, letterCount[i]);
}

putc ('\n', ofp);

fclose (ifp);                ← remember to close !!
fclose (ofp);

return 0;
}
```

- Single characters are typically printed with **putc**
- More complex expressions can be printed using **fprintf** (similar to **printf**)
- Similarly, **fscanf** can be used to read from file

File read / write – example

- Compile previous program into a file called `countCaps`, and run:

```
==> countCaps \  
    /share/classes/class02/a-tale-of-two-cities-full.txt \  
    letters.out  
  
==> cat letters.out
```

```
A: 895| B: 533| C: 708| D: 986| E: 402| F: 436|  
G: 385| H: 885| I:2870| J: 280| K:  55| L: 831|  
M:1625| N: 412| O: 338| P: 497| Q:  27| R: 214|  
S: 895| T:1691| U:  77| V: 115| W: 607| X:  60|  
Y: 438| Z:   0|
```